

El Maharajá y el Programador: ¿Quién Supervisa al Supervisor?

Luisa Fernanda Muñoz Calle

Código: 1037663395

Curso: Programación

Universidad CES

31 de julio de 2026

Resumen

El Dr. Raj Reddy, pionero de la inteligencia artificial, propone que la llegada de la Inteligencia Artificial General (AGI) hacia 2029-2030 inaugurará una era de abundancia cognitiva donde el aprendizaje tradicional será transformado por tutores de IA y el modelo de *Aprender Justo a Tiempo* (JIT). Este ensayo argumenta que, si bien el JIT es valioso como herramienta instrumental, resulta insuficiente como eje vertebrador de la educación superior en computación. Se sostiene que la habilidad meta-cognitiva de *Aprender a Aprender* —específicamente la capacidad de formular problemas, razonar algorítmicamente y verificar críticamente soluciones— trasciende la sintaxis de los lenguajes de programación y constituye el fundamento indispensable para que los futuros profesionales ejerzan el rol de supervisores estratégicos. La tesis central es que la verdadera competencia en la era de la IA no radica en saber programar, sino en saber *qué* programar y, más importante aún, en saber evaluar críticamente lo que la IA programa.

1. Introducción

En su conferencia *The Future of AI: Doomers vs. Abundance*, el Dr. Raj Reddy —ganador del Premio Turing y pionero de la inteligencia artificial desde la década de 1960— plantea una visión profundamente transformadora del futuro educativo [1]. Su tesis central es que la llegada de la Inteligencia Artificial General (AGI) hacia 2029-2030

redefinirá radicalmente la relación entre el ser humano y el conocimiento, inaugurando una *era de abundancia* donde la capacidad cognitiva será democratizada y el trabajo, eventualmente, opcional.

Ante este escenario, emerge una pregunta educativa fundamental: ¿Debe la educación en computación seguir enfatizando la disciplina del *Aprender a Aprender* —con su énfasis en fundamentos teóricos y práctica deliberada— o debe transformarse hacia un modelo de consumo ágil y consulta bajo demanda, un *Aprender Justo a Tiempo* (JIT) donde la IA actúa como tutor personalizado?

La postura que defenderé en este ensayo es que **el aprendizaje JIT, si bien valioso como herramienta instrumental, es insuficiente como eje vertebrador de la educación superior en computación**. Sostendré que la habilidad meta-cognitiva de *aprender a aprender* —específicamente la capacidad de formular problemas, razonar algorítmicamente y verificar críticamente soluciones— no solo trasciende la sintaxis de los lenguajes de programación, sino que constituye el fundamento indispensable para que los futuros profesionales ejerzan el rol de *supervisores estratégicos* que Reddy vislumbra. La tesis que guiará mi argumentación es que **la verdadera competencia en la era de la IA no radica en saber programar, sino en saber qué programar y, más importante aún, saber evaluar críticamente lo que la IA programa**.

2. El Mito del Sintactismo vs. la Meta-Cognición

La afirmación de Reddy de que los Modelos de Lenguaje de Gran Escala (LLMs) han vuelto irrelevante la sintaxis de los lenguajes de programación podría interpretarse, erróneamente, como un llamado a abandonar la enseñanza de la programación. Sin embargo, esta lectura superficial omite una distinción crucial que el propio Reddy enfatiza.

El *aprender a aprender* que Reddy defiende no se refiere a la memorización de sintaxis, sino a la capacidad de formular las preguntas correctas y estructurar problemas de manera lógica. Como señala en su conferencia, el verdadero desafío educativo es doble: *primero es hacer las preguntas correctas y segundo es que no sabes dar respuesta pero necesitas un aprendizaje personalizado para aprender*. Esta distinción es fundamental, pues sitúa el centro del aprendizaje no en la codificación, sino en la conceptualización.

La historia de la pedagogía en ciencias de la computación respalda esta perspectiva. Reddy menciona un estudio significativo realizado en China, donde los estudiantes que aprendieron a partir de ejemplos resueltos —enfocados en comprender patrones y es-

estructuras subyacentes— recordaban un 50 % más que aquellos que recibieron instrucción convencional. Este hallazgo sugiere que la verdadera retención del conocimiento proviene no de la memorización de reglas sintácticas, sino de la internalización de estructuras conceptuales profundas.

Por tanto, la *irrelevancia* de la sintaxis que Reddy señala no es un argumento contra el aprendizaje fundamental, sino contra el aprendizaje superficial. Los LLMs pueden generar código sintácticamente correcto, pero la capacidad de evaluar si ese código implementa la solución correcta, si su complejidad algorítmica es apropiada, o si sus abstracciones son lógicamente sólidas, sigue siendo una habilidad exclusivamente humana que exige un dominio conceptual que solo se desarrolla mediante el estudio deliberado y profundo.

3. La Ilusión del Aprendizaje Justo a Tiempo (JIT)

El modelo de aprendizaje *Justo a Tiempo* que Reddy describe —donde un estudiante podría aprender cálculo diferencial en una semana o un mes, según la necesidad inmediata— parece atractivo en su eficiencia aparente. Sin embargo, esta aproximación conlleva riesgos cognitivos significativos que Reddy no aborda directamente, pero que son implícitamente reconocibles en su énfasis en el rol de supervisor humano.

3.1. La ilusión de entendimiento

El primer riesgo es la *ilusión de entendimiento*. Cuando un estudiante utiliza un LLM para generar código funcional que resuelve un problema, puede experimentar la satisfacción de haber *completado* la tarea sin haber comprendido los principios subyacentes. Esta ilusión es particularmente peligrosa porque la funcionalidad superficial del código genera una falsa sensación de competencia. El estudiante cree dominar la ingeniería de software porque el código compila y produce resultados aparentemente correctos, pero carece de la comprensión necesaria para evaluar sus limitaciones, su eficiencia computacional (complejidad $O(n)$), o sus posibles efectos secundarios.

3.2. La fragmentación cognitiva

El segundo riesgo es cognitivo y epistemológico. El aprendizaje JIT, al fragmentar el conocimiento en unidades de consulta puntual, impide la formación de las estructuras mentales integradas que caracterizan el dominio experto. La comprensión profunda en

ciencias de la computación no es una colección de hechos aislados, sino un entramado conceptual donde los principios de diseño de algoritmos, las estructuras de datos, la complejidad computacional y la arquitectura de sistemas se relacionan de manera orgánica. Este entramado solo se desarrolla mediante la exposición sostenida y la práctica deliberada, no mediante consultas episódicas.

3.3. La incapacidad de supervisión

El tercer riesgo es pragmático. Reddy señala que la IA podrá programar, pero *necesitará alguien que le indique qué hacer y que vea que está bien y cómo escribir*. Esta función de evaluación y supervisión exige precisamente ese entendimiento profundo que el aprendizaje JIT exclusivo no proporciona. Un supervisor que solo sabe consultar pero no comprende no puede cumplir su función crítica.

4. La Fuerza Laboral del Futuro y el *Modelo Maharajá*

Reddy propone el *Modelo Maharajá* como una visión del futuro donde, gracias a la abundancia generada por la IA, el trabajo será opcional y los humanos podrán dedicarse a la reflexión y la creatividad. En este modelo, el ser humano asume un rol de supervisor estratégico, no de ejecutor técnico. Esta visión, aunque optimista, plantea una paradoja educativa que Reddy no resuelve explícitamente: **para supervisar efectivamente, el supervisor debe comprender lo que supervisa.**

La imposibilidad de verificar lo que no se comprende es un principio epistemológico fundamental que adquiere relevancia crítica en el contexto de sistemas de IA generativa. Los LLMs, como Reddy reconoce implícitamente al mencionar las *alucinaciones*, pueden producir resultados que parecen plausibles pero son incorrectos, o incluso peligrosos. Un supervisor que carece de fundamentos conceptuales sólidos no puede distinguir entre una solución algorítmicamente elegante y una que funciona por casualidad, entre un código robusto y uno frágil, entre una arquitectura escalable y una que colapsará bajo carga.

Consideremos el caso de sistemas críticos: software médico que debe tomar decisiones diagnósticas, sistemas bancarios que gestionan transacciones financieras, o infraestructura de transporte autónomo. En estos dominios, la incapacidad del supervisor para comprender los fundamentos conceptuales del sistema que supervisa no es meramen-

te un déficit educativo; es un riesgo existencial. Reddy, al señalar que *una máquina sepa programar necesitará alguien que le indique qué hacer*, está reconociendo que la agencia humana sigue siendo necesaria, pero no explica cómo un humano sin formación conceptual profunda podría ejercer esa agencia de manera efectiva.

La analogía con la medicina es ilustrativa: un médico no necesita saber fabricar los fármacos que receta, pero debe comprender su mecanismo de acción, sus contraindicaciones y sus efectos secundarios. De manera análoga, un supervisor de sistemas de IA debe comprender los principios fundamentales de la computación, no para programar, sino para evaluar críticamente lo que la IA produce.

5. Síntesis Filosófica y Veredicto Pedagógico

La paradoja educativa que emerge de la visión de Reddy puede resolverse mediante una integración jerárquica de modelos educativos. Mi propuesta es que los planes de estudio universitarios deben estructurarse según tres niveles de profundidad:

5.1. Nivel Fundamental: *Aprender a Aprender*

Este nivel debe constituir la columna vertebral de la formación en computación. Incluye: estructuras de datos y algoritmos, complejidad computacional, arquitectura de computadores, principios de diseño de software, lógica matemática y fundamentos de la teoría de la computación. La meta de este nivel no es la memorización de sintaxis, sino el desarrollo de un pensamiento computacional riguroso que permita formular problemas, razonar sobre soluciones y evaluar críticamente implementaciones. Reddy respalda implícitamente este énfasis cuando señala que *las cosas que requieren mucha práctica* son precisamente aquellas que la IA no puede reemplazar.

5.2. Nivel Instrumental: Aprendizaje Justo a Tiempo

Este nivel debe incorporarse como herramienta complementaria para la adquisición de habilidades técnicas específicas y actualización continua. El JIT es apropiado para aprender nuevos lenguajes de programación, bibliotecas específicas, o tecnologías emergentes, siempre que el estudiante posea ya los fundamentos conceptuales para evaluar críticamente lo que aprende. Reddy ejemplifica esta aproximación con el aprendizaje del cálculo: *ahora cálculo puedes aprenderlo en 1 día o 1 semana, un mes y el resto cuando lo necesites*, pero esta afirmación presupone que el estudiante ya posee la madurez

matemática para comprender los conceptos.

5.3. Nivel Práctico: Aprendizaje Haciendo

La pedagogía que Reddy describe —*un día te dirá aquí está el trabajo para hoy descúbrela tú solo aprendiendo haciendo*— debe implementarse como un espacio donde los estudiantes aplican sus fundamentos conceptuales a problemas auténticos, utilizando herramientas de IA como asistentes, no como sustitutos del pensamiento crítico.

5.4. Recomendaciones para la Educación Superior

A continuación, se presentan cinco recomendaciones concretas para la educación superior en computación:

1. **Mantener los cursos fundamentales** (algoritmos, estructuras de datos, teoría de la computación) como requisitos obligatorios, pero actualizando su enfoque para enfatizar el razonamiento conceptual sobre la implementación sintáctica.
2. **Incorporar la IA generativa como herramienta pedagógica** desde los primeros cursos, no para reemplazar el aprendizaje, sino para permitir que los estudiantes se enfoquen en problemas de mayor nivel conceptual mientras la IA se encarga de la implementación rutinaria.
3. **Desarrollar competencias de evaluación crítica** mediante ejercicios donde los estudiantes deben identificar errores, alucinaciones y limitaciones en código y soluciones generadas por IA.
4. **Fomentar la metacognición** mediante la reflexión explícita sobre el proceso de aprendizaje: los estudiantes deben documentar no solo lo que aprendieron, sino *cómo* aprendieron y *cómo* verificaron la corrección de sus soluciones.
5. **Rediseñar la evaluación** para medir no la capacidad de reproducir código, sino la capacidad de formular problemas, evaluar soluciones y argumentar sobre decisiones de diseño y complejidad.

6. Conclusión

La visión de Raj Reddy sobre la era de la abundancia cognitiva no es una invitación a abandonar el rigor educativo, sino un llamado a redefinirlo para una era donde la

capacidad de generar código será ubicua, pero la capacidad de evaluarlo críticamente será más valiosa que nunca. La verdadera competencia en este nuevo paradigma no radica en la memoria sintáctica ni en la velocidad de codificación, sino en la profundidad conceptual que permite a un ser humano ejercer el rol de supervisor estratégico que Reddy vislumbra.

El aprendizaje *Justo a Tiempo* tiene un lugar legítimo como herramienta instrumental para la actualización continua y la adquisición de habilidades específicas. Pero no puede, ni debe, reemplazar el cultivo deliberado del pensamiento computacional profundo que solo se desarrolla mediante el estudio sistemático y la práctica reflexiva. Como sugiere la metáfora del *Modelo Maharajá*, la humanidad está llamada a un rol de supervisión y dirección que exige, paradójicamente, una preparación más rigurosa y profunda, no menos.

La educación superior en computación debe, por tanto, abrazar la era de la abundancia digital no abandonando sus fundamentos, sino reivindicándolos como la base indispensable para que los futuros profesionales puedan navegar, supervisar y dirigir los sistemas de inteligencia artificial que están transformando nuestro mundo. El desafío no es aprender menos, sino aprender de manera diferente: no para hacer lo que la máquina puede hacer, sino para poder dirigir lo que la máquina *debe* hacer.

Referencias

- [1] Reddy, R. (2026). *The Future of AI: Doomers vs. Abundance* [Video]. Carnegie Mellon University. <https://www.youtube.com/watch?v=ydnU5MbyyQo>
- [2] Reddy, R. (2026). Biography and Research Interests. *Portal of Robotics and Artificial Intelligence*. <https://pr.ai/threads/raj-reddy.27840/>